Lecture 11 — REST API Design

Backend Engineering from First Principles

These notes are based on the supplied lecture transcript. The lecture focuses on designing REST API interfaces: resource naming, routes, HTTP methods, status codes, pagination, sorting, filtering, custom actions, and consistency.

1. Why API Design Matters

Backend engineers spend a significant amount of time designing APIs. An API is the interface through which clients—such as a browser frontend, mobile app, or another server—interact with backend functionality.

This lecture focuses on **REST APIs**. Other API approaches mentioned include:

- RPC (Remote Procedure Call)
- GraphQL

The goal is not to invent a new standard. It is to follow established conventions so that:

- API behavior is predictable.
- Frontend and backend engineers make fewer assumptions.
- Integrations require less guesswork.
- Documentation is easier to understand.
- There are fewer avoidable bugs and misunderstandings.
- Engineers can focus on business logic instead of repeatedly debating naming and HTTP semantics.

A good API interface should be intuitive, consistent, and clear to its consumers.

2. A Little History: Where REST Came From

The lecture gives historical context for the web and REST:

1. In 1990, Tim Berners-Lee started the World Wide Web project to make sharing knowledge and information easier.

2. Early web foundations included URIs, HTTP, HTML, a web server, a web browser, and an HTML editor.

3. As the web grew rapidly, scalability became an important concern.

4. Roy Fielding proposed architectural constraints to help the web scale.

5. Fielding later described the architectural style in his 2000 PhD dissertation and named it **REST — Representational State Transfer**.

The lecture recommends reading Roy Fielding's REST dissertation for deeper historical and architectural context.

3. REST Architectural Constraints

The lecture discusses six constraints associated with REST.

3.1 Client–Server

Separate responsibilities between client and server.

• Client: user interface and user experience.

• Server: data storage and business logic.

This separation allows both sides to evolve independently.

3.2 Uniform Interface

Components communicate through a consistent, standardized interface.

The lecture identifies four sub-constraints:

1. Resource identification.

2. Resource manipulation through representations.

3. Self-descriptive messages.

4. Hypermedia as the engine of application state (HATEOAS).

The key idea is that a consistent interface makes the system easier to understand and integrate.

3.3 Layered System

The architecture can contain hierarchical layers. A layer interacts with the appropriate adjacent layer rather than needing to know every internal detail of the system.

Examples of intermediate components include:

• Load balancers.

• Proxy servers.

• Other infrastructure layers.

This supports scalability and separation of concerns.

3.4 Cacheability

Responses should explicitly indicate whether they can be cached.

Caching can:

• Reduce repeated work on the server.

• Reduce network traffic.

• Improve response times.

• Improve the user experience.

3.5 Statelessness

Each request should contain the information needed by the server to understand and process that request.

The server should not depend on remembering the client's previous request context as part of the REST stateless constraint.

Why this helps:

• Any suitable server can handle an incoming request.

• Load balancers can distribute requests across multiple servers.

• The system becomes easier to scale horizontally.

3.6 Code on Demand (Optional)

A server can optionally transfer executable code—such as JavaScript—to extend client functionality.

The lecture emphasizes that this constraint is optional.

4. What Does REST Mean?

REST stands for **Representational State Transfer**.

Representational

A resource can be represented in different formats depending on the communication context.

Examples:

• JSON.

• XML.

• HTML.

The same user resource could be represented as JSON for an API client and as HTML for a browser-oriented response.

State

State refers to the current condition or attributes of a resource.

Example: a shopping cart's state may include:

• Items in the cart.

• Quantity of each item.

• Total price.

Transfer

The representation of a resource moves between client and server. HTTP provides the communication protocol and methods used for this exchange.

Putting the three words together

REST describes an architectural style in which:

• Resources have representations.

• Resource state is expressed through those representations.

• Clients and servers transfer representations using a uniform interface and agreed constraints.

5. Understanding a URL

A typical URL can contain these parts:

```
https://www.example.com/books?sort=name#details
|---|   |--------------| |----| |------|
scheme      authority    query  fragment
                         params
```

Part	Meaning
Scheme	Protocol, such as `http` or `https`
Authority	Host/domain, optionally including subdomain and port
Path	Resource or route being accessed
Query parameters	Key-value information such as filters, sorting, and pagination
Fragment	Browser-side reference to a section of a page

The forward slash in a path expresses a hierarchical relationship between path segments.

Typical API URL structure

The lecture's illustrative structure is:

```
https://api.example.com/v1/books
        |      |       |  |
      subdomain version  resource
```

Common parts:

1. HTTPS scheme.

2. `api` subdomain.

3. API version, such as `v1`.

4. Resource path.

Versioning through the route is a common pattern discussed in the lecture, not a mandatory requirement for every API.

6. Resource Naming Rules

6.1 Use plural nouns for resource paths

The lecture recommends using plural nouns for resource path segments.

Prefer:

```
GET /v1/books
GET /v1/books/123
```

Avoid switching between singular and plural:

```
GET /v1/book
GET /v1/books/123
```

Even when retrieving one book, the path represents an item belonging to the `books` resource collection.

6.2 Use lowercase path segments

Prefer lowercase resource names:

```
/v1/organizations
```

Avoid inconsistent capitalization in routes.

6.3 Avoid spaces and underscores in URL paths

For human-readable phrases used in paths, the lecture recommends:

1. Convert the phrase to lowercase.

2. Replace spaces with hyphens.

Example:

```
Harry Potter
      |
      v
harry potter
      |
      v
harry-potter
```

The resulting slug is easier to read in a URL:

```
/books/harry-potter
```

6.4 Think hierarchically

For example:

```
/organizations
/organizations/{organizationId}
/organizations/{organizationId}/projects
/organizations/{organizationId}/projects/{projectId}
```

Read this as:

• A collection of organizations.

• One organization inside that collection.

• Projects belonging to that organization.

• One project belonging to that organization.

Use nested paths when the relationship is meaningful to the API.

7. Idempotency

Idempotency is the property of an operation where performing the same action multiple times has the same intended effect on server state as performing it once.

In API design, the important point is the side effect on the server—not necessarily whether every response body is identical.

Example:

• Setting a user's name to `A` and then setting it to `B` changes the state.

• Repeating the same update that sets the name to `B` leaves the name as `B`.

HTTP methods and idempotency in the lecture

Method	Typical purpose	Idempotent?	Why
GET	Retrieve data	Yes	Does not change server state
POST	Create resource / custom action	Usually no	Repeated calls may create repeated effects
PUT	Replace a resource representation	Yes	Repeating the same replacement has the same intended state
PATCH	Partially update a resource	Usually yes for the lecture's same-value update example	Repeating the same field update leaves the same final value
DELETE	Delete a resource	Yes	After the first deletion, repeating it does not create a new state change

Important distinction: A repeated GET can return different data if the underlying resource changes between requests. The idempotency concept concerns the request's effect on server state, not a guarantee that the response will never change.

The lecture uses the five common methods above and also mentions `HEAD` and `OPTIONS`.

8. Choosing the Correct HTTP Method

GET — Read

Use GET to retrieve a resource or collection.

```
GET /v1/organizations
GET /v1/organizations/{organizationId}
```

GET should not be used to modify server state.

POST — Create

Use POST when creating a new resource.

```
POST /v1/organizations
```

Example request body:

```
{
  "name": "Arc One",
  "status": "active",
  "description": "An example organization"
}
```

The server generates server-managed fields such as the ID and timestamps.

The lecture's typical create response is **201 Created**, often with the newly created resource in the response body.

POST is also used for custom actions that do not fit the ordinary CRUD operations.

PATCH — Partial update

Use PATCH when updating only selected fields of a resource.

```
PATCH /v1/organizations/{organizationId}
```

Example:

```
{
  "status": "active"
}
```

Only the supplied field is being changed in the lecture's example.

PUT — Replace

Use PUT when replacing the resource representation with the supplied representation.

The lecture's conceptual example sends the complete representation rather than only the changed field.

DELETE — Delete

Use DELETE to remove a resource.

```
DELETE /v1/organizations/{organizationId}
```

The lecture's typical successful delete response is **204 No Content**, with an empty response body.

9. CRUD API Design Pattern

CRUD means:

• **C**reate

• **R**ead

• **U**pdate

• **D**elete

A typical resource endpoint set looks like this:

Operation	Method	Route
List organizations	GET	`/v1/organizations`
Create organization	POST	`/v1/organizations`
Get one organization	GET	`/v1/organizations/{id}`
Partially update organization	PATCH	`/v1/organizations/{id}`
Delete organization	DELETE	`/v1/organizations/{id}`

Notice:

• List and create use the collection route.

• Get-one, update, and delete use the resource route with an ID.

• The HTTP method distinguishes the operation when the path is the same.

Server-managed fields

In the lecture's create example, the client sends fields such as:

• `name`

• `status`

• `description`

The server/database handles fields such as:

• `id`

• `createdAt`

• `updatedAt`

Do not ask clients to supply values that should be generated and controlled by the server.

10. HTTP Status Codes for Common API Outcomes

Scenario	Status	Meaning
Successful fetch	200 OK	Request succeeded and response contains the result
Successful update	200 OK	Update succeeded; updated data may be returned
Resource created	201 Created	A new resource was created
Successful delete with no body	204 No Content	Operation succeeded; no response body
Requested individual resource does not exist	404 Not Found	The requested resource was not found
Too many requests	429 Too Many Requests	Client exceeded a rate limit
Invalid request	400 Bad Request	Client request is invalid
Unexpected server failure	500 Internal Server Error	Server failed to process the request

Important: 404 for an item vs empty list

The lecture emphasizes this distinction:

Single resource request

```
GET /v1/organizations/unknown-id
```

If that organization does not exist, return **404 Not Found**.

List request

```
GET /v1/organizations?status=archived
```

If no organizations match, return a successful list response with an empty array, usually **200 OK**.

Example:

```
{
  "data": [],
  "total": 0,
  "page": 1,
  "totalPages": 0
}
```

A list endpoint is still a successful query even when the result set is empty.

11. Pagination

Pagination is a technique for returning a portion of a large collection instead of sending every record in one response.

Why use it?

• Serializing a very large collection can be expensive.

• Large responses consume more network bandwidth.

• Clients may take longer to receive and render the data.

• Users often see only a small portion of a list at a time.

Common query parameters

The lecture demonstrates:

• `limit`: maximum number of records in one response.

• `page`: which page of results to return.

Example:

```
GET /v1/organizations?limit=10&page=1
GET /v1/organizations?limit=10&page=2
```

The first request returns the first portion; the second request returns the next portion.

Typical pagination response

```
{
  "data": [
    {
      "id": "org_123",
      "name": "Arc One"
    }
  ],
  "total": 42,
  "page": 1,
  "totalPages": 5
}
```

Field	Meaning
`data`	Records in the current page
`total`	Total matching records
`page`	Current page number
`totalPages`	Number of pages available

The lecture recommends choosing sensible default values for `limit` and `page` when the client does not send them.

12. Sorting

Sorting lets the client choose the field used to order a list.

The lecture demonstrates query parameters such as:

```
GET /v1/organizations?sortBy=name&sortOrder=asc
```

Common concepts:

- `sortBy`: field used for sorting.
- `sortOrder`: direction, such as ascending or descending.

The lecture's example describes descending as the default in its demo and shows ascending as an explicit option.

The API should validate allowed sort fields and sort directions rather than blindly accepting arbitrary client input.

13. Filtering

Filtering lets a client request only records that match particular criteria.

Example:

```
GET /v1/organizations?status=archived
```

This asks the server to return organizations whose status is `archived`.

Another request:

```
GET /v1/organizations?status=active
```

asks for active organizations.

Filtering is especially useful for list endpoints that power search, tabs, dropdowns, and filter controls in a frontend.

If no records match a filter, return an empty list response rather than treating the list itself as a missing resource.

14. Custom Actions with POST

Not every server operation is a simple create, read, update, or delete.

A **custom action** is an operation that represents a business action and may trigger several server-side steps.

The lecture's examples are:

Archive an organization

Archiving may involve more than changing a status field:

• Updating the organization's status.

• Processing projects associated with it.

• Processing tasks associated with those projects.

• Sending notifications or emails.

Because the action represents a larger business operation, the lecture models it as a custom action:

```
POST /v1/organizations/{organizationId}/archive
```

Clone a project

Cloning a project may involve:

• Creating a new project.

• Copying project fields.

• Cloning associated tasks.

• Triggering additional server-side work or notifications.

Example:

```
POST /v1/projects/{projectId}/clone
```

Why POST?

The lecture presents POST as the open-ended method for actions that do not fit the standard CRUD categories.

Do not assume every POST returns 201

• If POST creates a new resource, the response may be **201 Created**.

• If a custom action succeeds without creating a resource, it may return **200 OK**.

• If a custom action creates a new resource (such as a cloned project), it may return **201 Created**.

Choose the status code based on what actually happened on the server, not only based on the HTTP method.

15. API Payload and Naming Consistency

The lecture strongly emphasizes consistency across all endpoints in one API.

JSON fields: use camelCase

The lecture recommends camelCase for JSON field names.

Example:

```
{
  "organizationId": "org_123",
  "createdAt": "2026-10-30T10:00:00Z",
  "description": "Project description"
}
```

Reuse the same field names

If one resource uses `description`, another resource with the same concept should also use `description`.

Avoid arbitrary alternatives such as:

• `description`

• `desc`

• `dsc`

Even if the names mean the same thing, inconsistent naming forces API consumers to remember exceptions.

Keep route conventions consistent

If organization routes use plural nouns and lowercase paths, project routes should follow the same convention.

Consistency should apply to:

• Resource paths.

• JSON field names.

• Query parameters.

• Success response shapes.

• Error response shapes.

• Documentation.

16. Designing an API: Recommended Workflow

The lecture recommends designing the API interface before writing the implementation.

Step 1 — Start with product requirements and UI wireframes

Study the UI designs, Figma wireframes, user stories, and requirements.

Ask:

• What does the user need to see?

• What actions can the user perform?

• What data does the frontend need?

• What information must the backend expose?

Step 2 — Identify resources from nouns

A useful starting heuristic is to identify nouns in the requirements.

For the lecture's project-management platform example:

Noun from requirements	Resource
Organization	`organizations`
Project	`projects`
Task	`tasks`
User	`users`
Tag	`tags`

The lecture calls this a thumb rule: nouns help reveal candidate resources. Requirements analysis is still needed to determine the actual resource model.

Step 3 — Design the database schema

The lecture briefly illustrates an example with:

• Organization table.

• Project table.

• Task table.

Database schema design is treated as a separate topic in the playlist.

Step 4 — List the actions clients need

For each resource, identify the operations clients need:

• Create.

• List.

• Fetch one.

• Update.

• Delete.

• Any additional business-specific actions.

Step 5 — Design the API interface

Define:

• HTTP method.

• Route.

• Path parameters.

• Query parameters.

• Request body.

• Success response.

• Error response.

• Status code.

The lecture uses **Insomnia** as an API design/testing tool and mentions Postman as an alternative.

Step 6 — Keep the design consistent

Apply the same route, payload, and response conventions across resources.

Step 7 — Provide interactive documentation

The lecture recommends interactive API documentation, mentioning Swagger tooling as an example.

Interactive documentation helps:

• API developers test endpoints.

• Frontend engineers understand request and response formats.

• Integrators explore the API without guessing.

• Teams maintain a shared contract.

17. Worked Example: Project Management API

Assume the platform has three primary resources:

• Organizations.

• Projects.

• Tasks.

Organization endpoints

Purpose	Method	Example route
List organizations	GET	`/v1/organizations`
Create organization	POST	`/v1/organizations`
Get organization	GET	`/v1/organizations/{organizationId}`
Update organization partially	PATCH	`/v1/organizations/{organizationId}`
Delete organization	DELETE	`/v1/organizations/{organizationId}`
Archive organization	POST	`/v1/organizations/{organizationId}/archive`

Project endpoints

Purpose	Method	Example route
List projects	GET	`/v1/projects`
Create project	POST	`/v1/projects`
Get project	GET	`/v1/projects/{projectId}`
Update project partially	PATCH	`/v1/projects/{projectId}`
Delete project	DELETE	`/v1/projects/{projectId}`
Clone project	POST	`/v1/projects/{projectId}/clone`

The lecture demonstrates pagination on project lists using `limit` and `page`, and describes sorting and filtering as common list endpoint features.

The same basic CRUD design pattern can be applied to tasks and other resources.

18. Quick Reference: API Design Rules

Concern	Lecture guideline
Resource names	Use plural nouns
Path casing	Use lowercase
Multiword slugs	Use hyphens instead of spaces/underscores
URL hierarchy	Use path segments to express resource relationships
Versioning	Commonly include a version such as `/v1`
Read	GET
Create	POST
Partial update	PATCH
Full replacement	PUT
Delete	DELETE
Custom action	POST
Create response	Commonly 201
Successful fetch/update	Commonly 200
Successful delete with no body	Commonly 204
Missing individual resource	404
Empty list result	200 with an empty array
List controls	Pagination, sorting, filtering
JSON naming	camelCase
Payloads and responses	Consistent field names and structure
Documentation	Provide interactive API documentation

19. Interview Questions and Answers

Q1. What does REST stand for?

Representational State Transfer.

Q2. What are the six REST constraints discussed in the lecture?

Client–Server, Uniform Interface, Layered System, Cacheability, Statelessness, and Code on Demand (optional).

Q3. What is a resource in REST API design?

A resource is an entity or concept exposed by the API, such as an organization, project, user, or task.

Q4. Why use plural nouns in API paths?

The path represents a resource collection, even when an individual item is accessed by appending its identifier.

Q5. What is idempotency?

Repeating an operation has the same intended effect on server state as performing it once.

Q6. Which HTTP methods are generally idempotent?

GET, PUT, and DELETE are idempotent. PATCH can be idempotent depending on the operation. POST is generally non-idempotent.

Q7. What is the difference between PUT and PATCH?

PUT is used to replace a resource representation; PATCH is used for a partial modification.

Q8. Why is POST used for custom actions?

Some business actions do not fit standard CRUD semantics. The lecture uses POST for such action endpoints.

Q9. What is the difference between 200, 201, and 204?

- 200: request succeeded.
- 201: a resource was created.
- 204: request succeeded and there is no response body.

Q10. When should an API return 404?

When the client requests a particular resource that does not exist. An empty list result is not normally a 404.

Q11. What is pagination?

Returning a subset of a collection per request rather than sending the entire collection at once.

Q12. What are `limit` and `page`?

`limit` controls the number of records in a page; `page` selects which page to retrieve.

Q13. What is the difference between sorting and filtering?

Sorting changes the order of results. Filtering restricts which results are returned.

Q14. Why is consistency important in API design?

It reduces guesswork, integration time, assumptions, and avoidable bugs for API consumers.

Q15. Why should API design happen before implementation?

It establishes the contract that clients will use and helps align routes, payloads, responses, and business requirements before code is written.

20. Quick Revision Sheet

• REST = Representational State Transfer.

• REST constraints: Client–Server, Uniform Interface, Layered System, Cacheability, Statelessness, optional Code on Demand.

• Resources are commonly identified from nouns in product requirements.

• Use plural, lowercase resource paths: `/v1/projects`.

• Use an ID to identify one item: `/v1/projects/{projectId}`.

• GET = read; POST = create/custom action; PATCH = partial update; PUT = replacement; DELETE = delete.

• Idempotency concerns the effect on server state.

• 200 = success; 201 = created; 204 = no content; 404 = missing individual resource.

• Empty list = 200 with `data: []`, not 404.

• Pagination = `limit` + `page`.

• Sorting = `sortBy` + `sortOrder`.

• Filtering = field-based query parameters such as `status=active`.

• Custom action examples: archive organization, clone project.

• JSON fields should follow camelCase in the lecture's convention.

• Keep route names, payloads, query parameters, and response structures consistent.

• Design the API interface before implementing business logic.

• Use interactive API documentation such as Swagger tooling.

Remember the design workflow

```
UI / Requirements
       |
       v
Identify resources (nouns)
       |
       v
Design database schema
       |
       v
```

```
List client actions (CRUD + custom)
       |
       v
Define routes, methods, params, payloads
       |
       v
Define status codes and response shapes
       |
       v
Document and test the API contract
       |
       v
Implement the backend
```